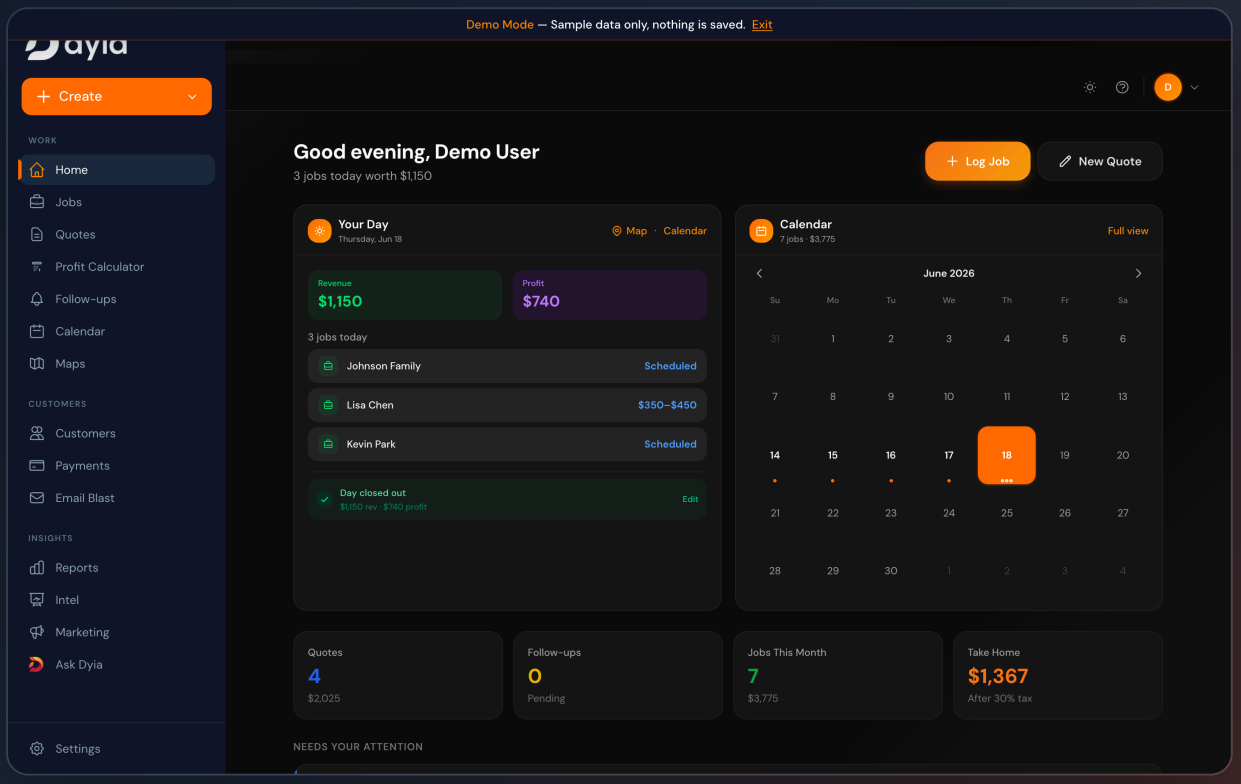
AI SAAS PLATFORM · TECHNICAL SHOWCASE

AI SaaS Platform Technical Showcase

Product walkthrough, system architecture, AI workflow design, and production readiness.

A production, multi-tenant SaaS for service businesses — job & revenue tracking, quoting with PDF export, a follow-up pipeline, integrated payments, and a tool-calling AI assistant that proposes actions for human approval before writing to the system of record.



Prepared by **Dev Ganugapenta / InitDev**

Platform: **dyia** — "Your Day, Decoded" · All screenshots captured live from the running application in Demo Mode (sample data only — no customer information).

Contents

Throughout this document: **IMPLEMENTED** = present and verifiable in the codebase **PARTIAL** = present in a related form **PROPOSED** = recommended extension, not built yet.

1 Executive Summary

The platform is an operations system for owner-operated service businesses (junk removal, lawn care, house cleaning). It replaces the spreadsheets and disconnected tools these operators use to run day-to-day work.

What problem it solves

Small service operators lose money to invisible costs and dropped leads: untracked expenses, quotes that are never followed up, and no clear picture of daily profit. The platform centralizes jobs, quotes, customers, follow-ups, payments, and reporting, then layers an AI assistant on top so the operator can run the business by asking for what they want instead of navigating forms.

Who it serves

Individual operators and small crews. The product is multi-tenant: every record is scoped to a user account, gated by subscription tier (**basic / trial / pro**), with an internal **admin** role for operations and support.

The core workflow

Capture a job or quote → track revenue and itemized expenses → compute live profit against a monthly goal → convert quotes to jobs and drive a follow-up pipeline → collect payment → review analytics. The AI assistant can perform most of these steps on request, but **mutating actions are proposed and require explicit human confirmation** before any record is written.

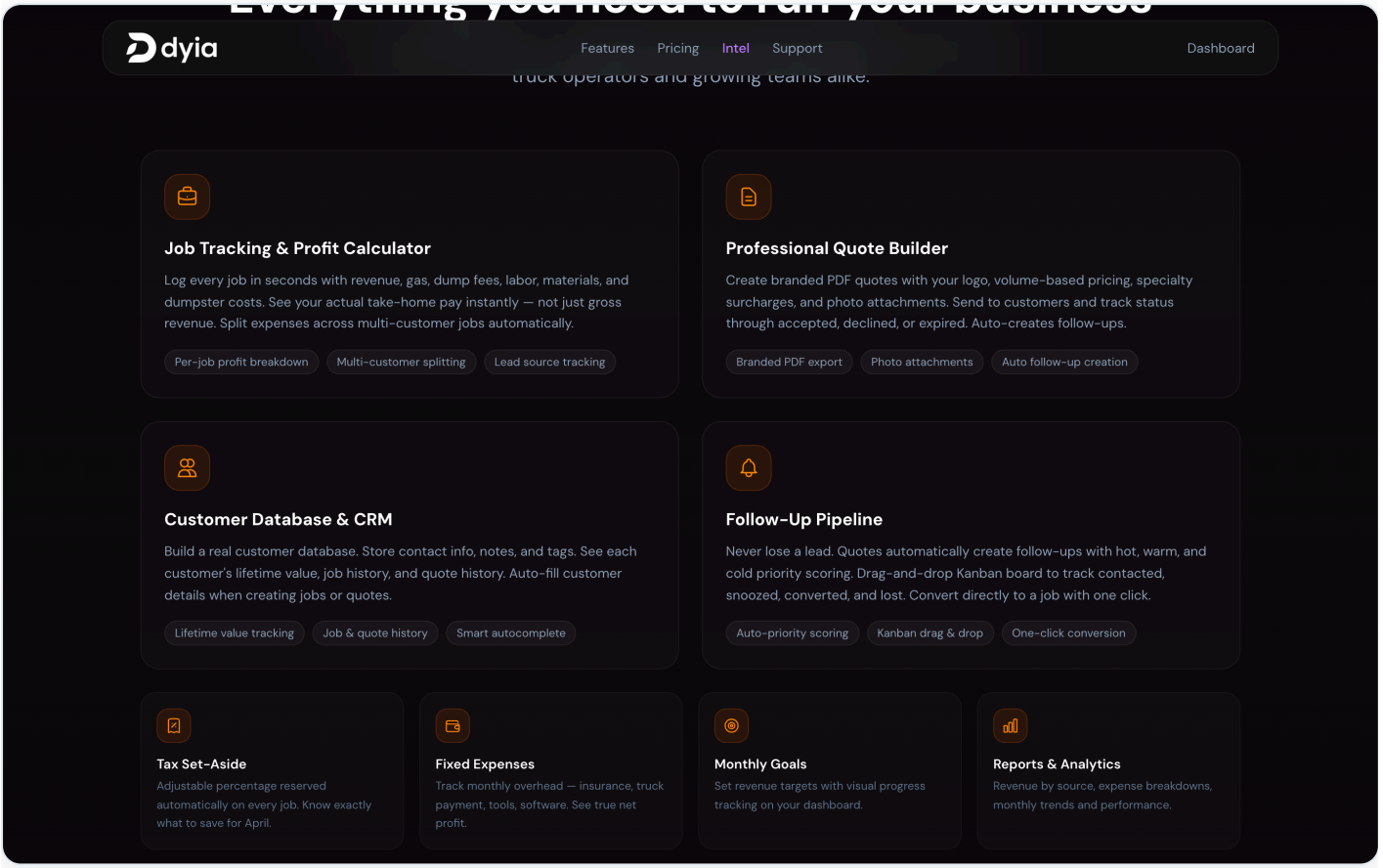
Why it is technically relevant

- **AI orchestration with tool-calling.** A stateful LLM layer with 21 strictly-typed tools, a bounded tool-call loop, and a propose/confirm pattern that keeps a human in control of writes.
- **Backend state management.** ~25 relational tables with explicit status enums, audit columns, and a persisted action queue.
- **Real third-party integrations.** Authentication, subscription billing, merchant payouts, geocoding, transactional email, error monitoring, and Redis-backed rate limiting.
- **Production SaaS hygiene.** Auth middleware, row-level security, webhook signature verification, scheduled jobs, and budget/rate guardrails on the AI layer.

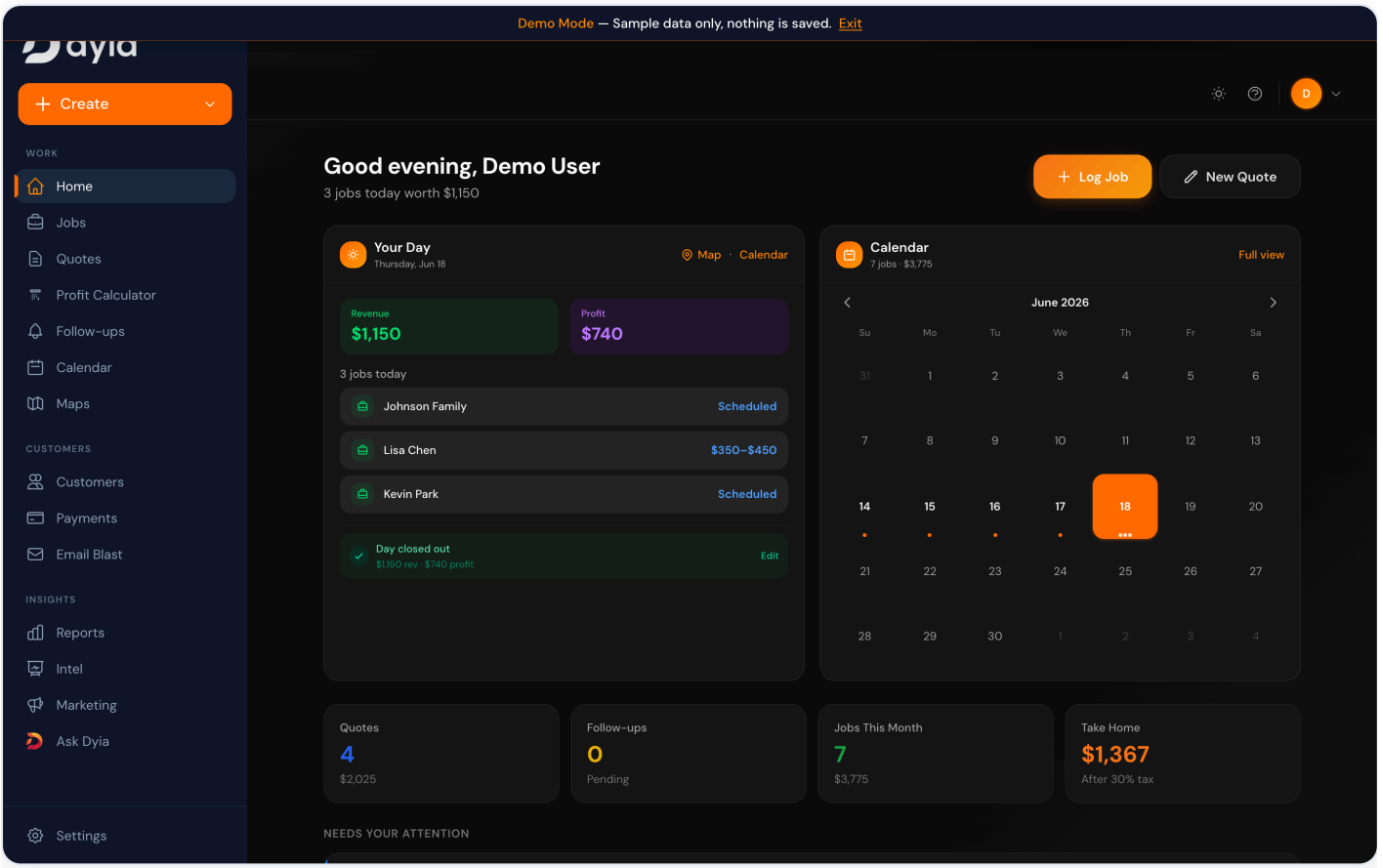
~69	21	~25	45+	4
API ROUTE HANDLERS	AI TOOLS (STRICT SCHEMA)	DATABASE TABLES	SQL MIGRATIONS	SCHEDULED CRON JOBS

2 Product Walkthrough

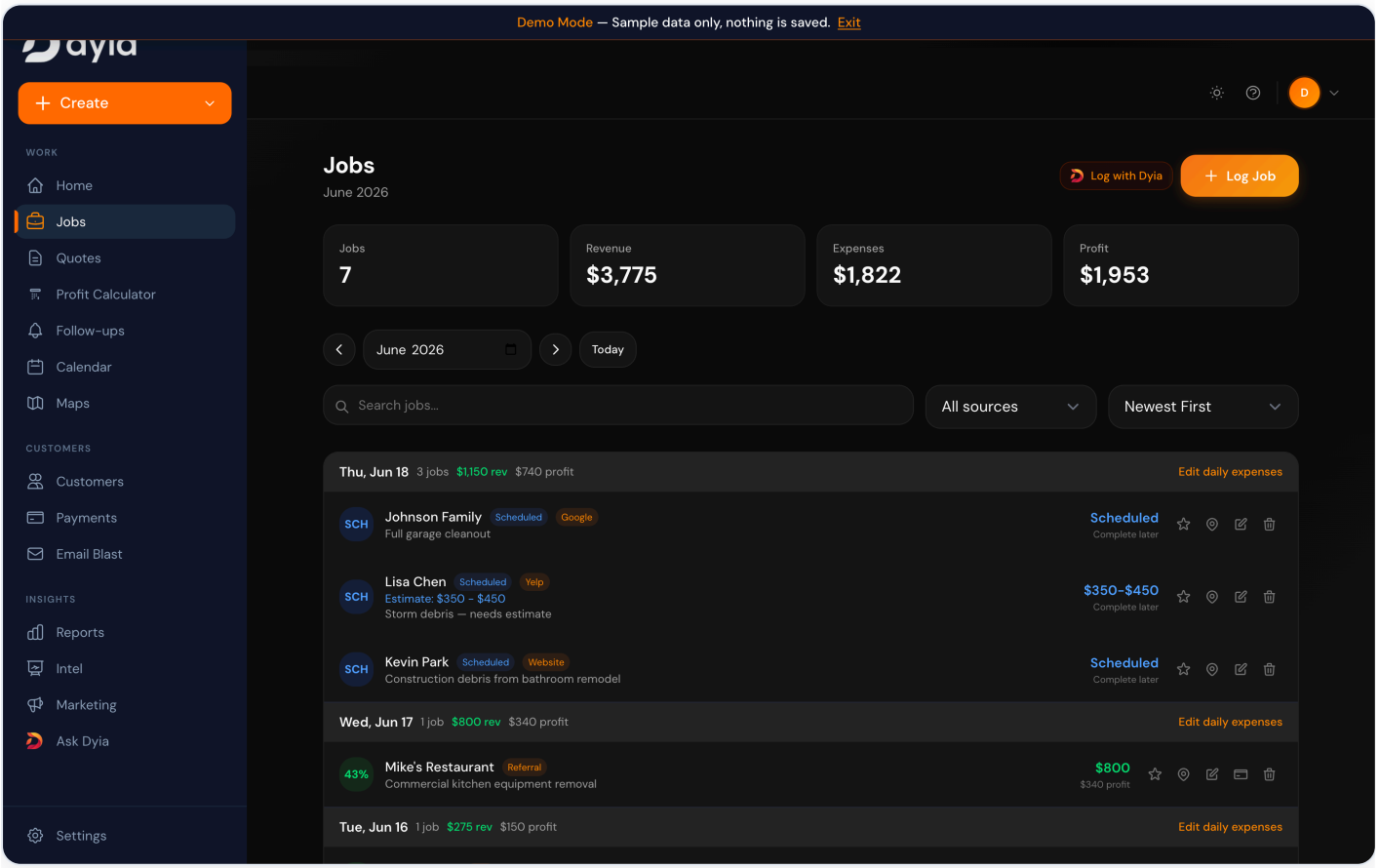
Captured live from the running application in Demo Mode. Each caption notes what the screen demonstrates from an engineering standpoint.



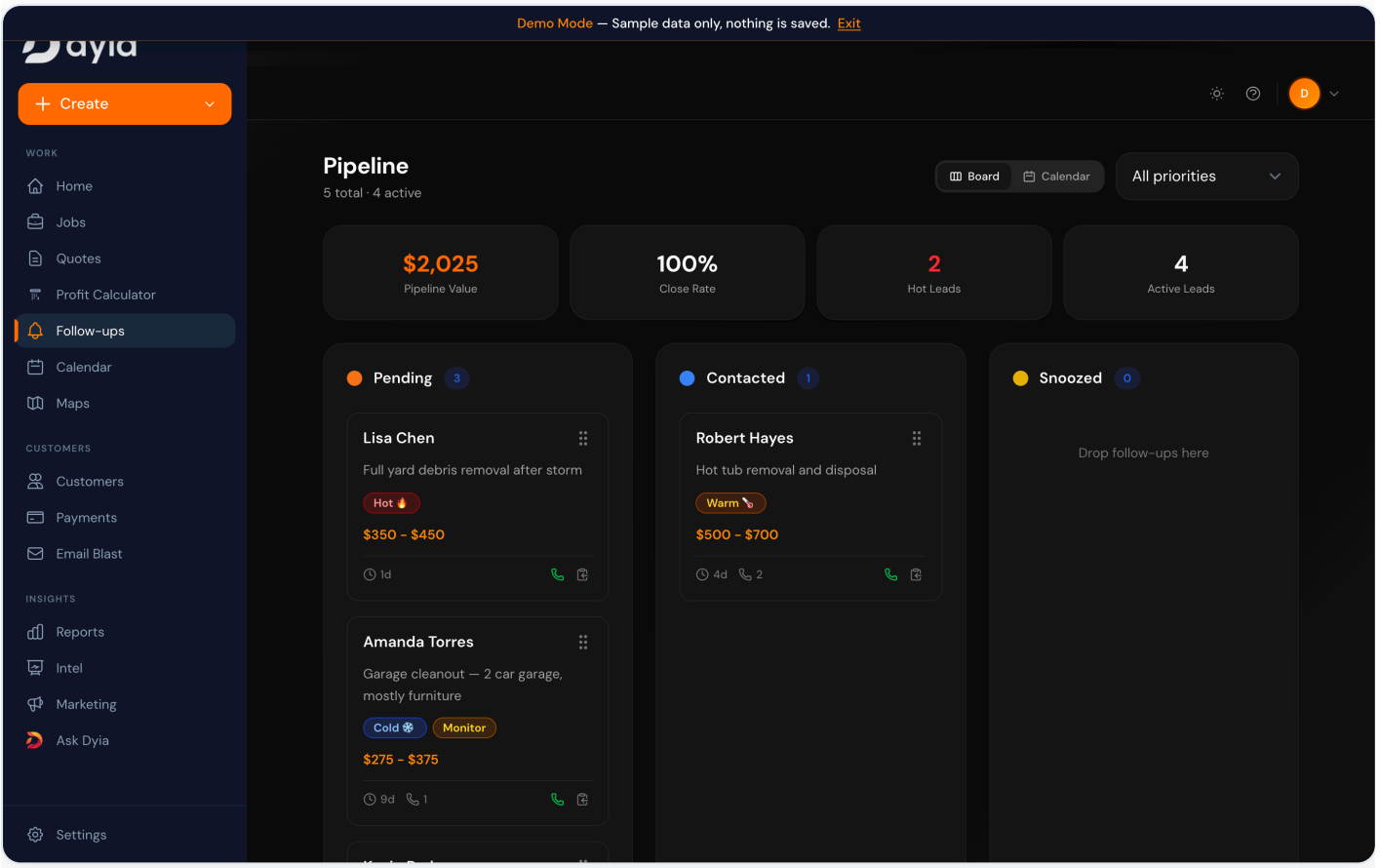
Public marketing surface with a self-serve demo path. **Proves:** a full public/authenticated split, SEO-oriented routing, and a real onboarding funnel — not a single-screen prototype.



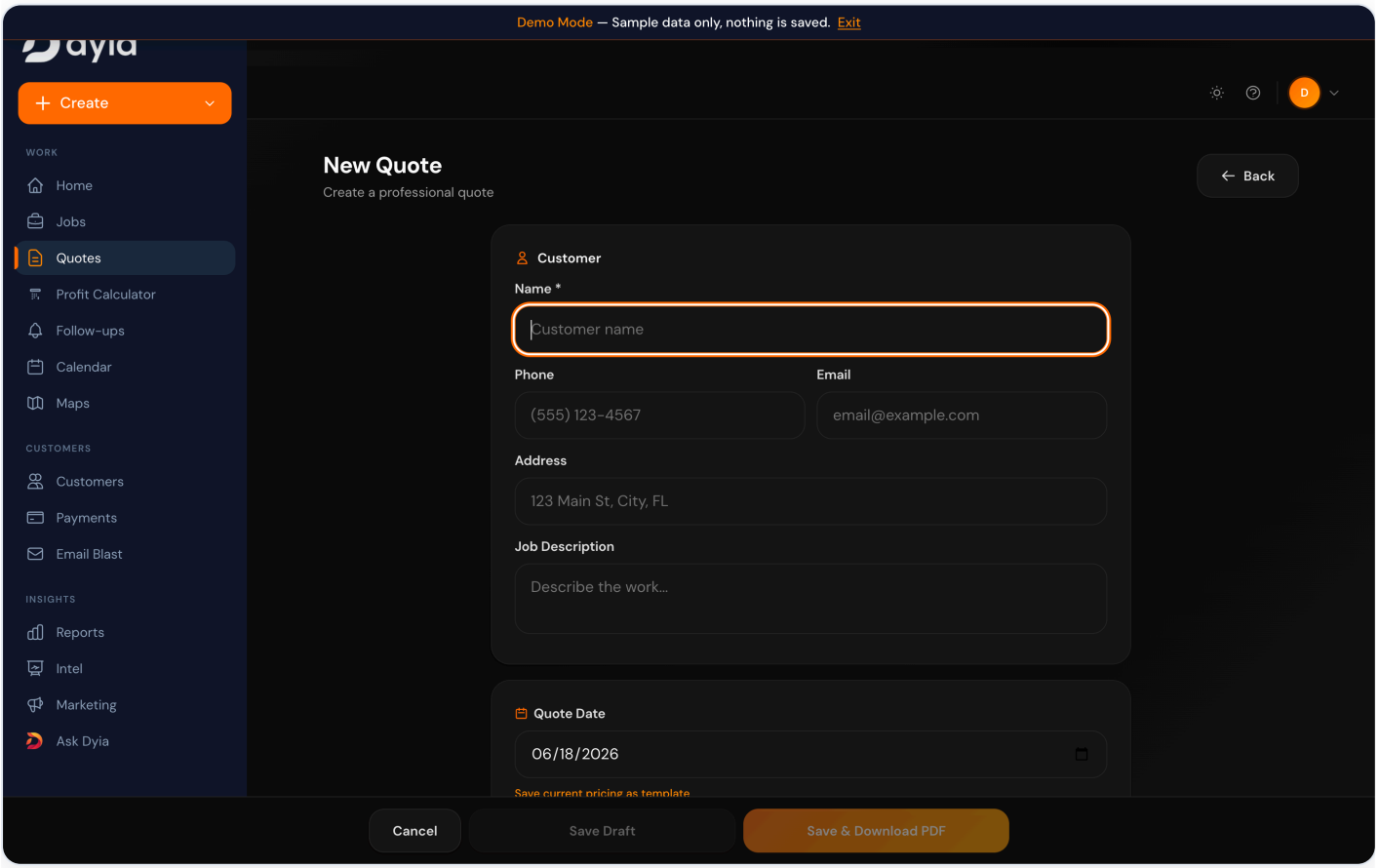
Authenticated home: today's schedule, live revenue/profit, monthly-goal progress, and a calendar. **Proves:** server-scoped data aggregation and real-time computed business state per tenant.



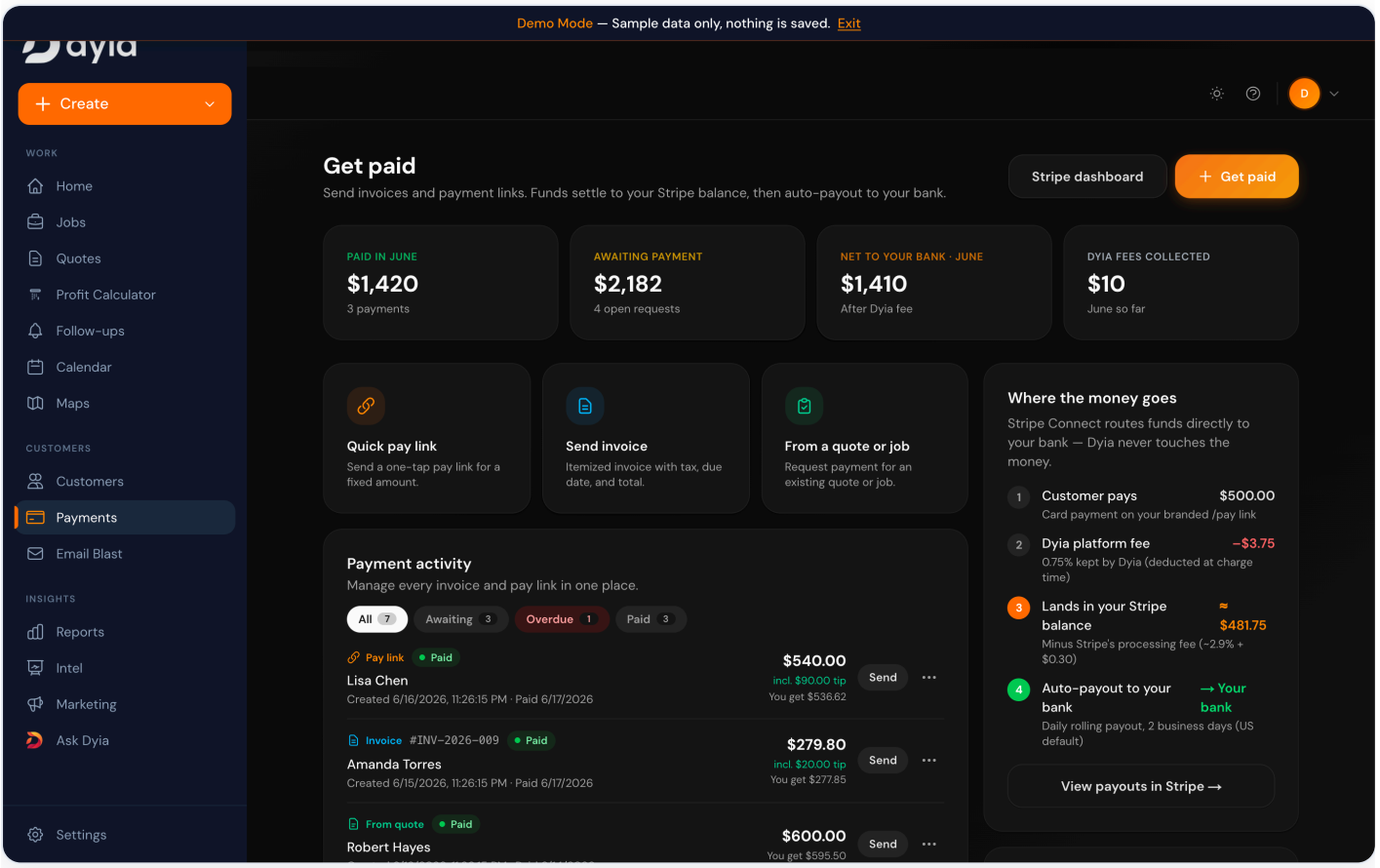
Job ledger with per-job revenue, itemized expenses, computed profit/margin, source attribution, and status. **Proves:** transactional CRUD plus financial computation, grouped and filterable.



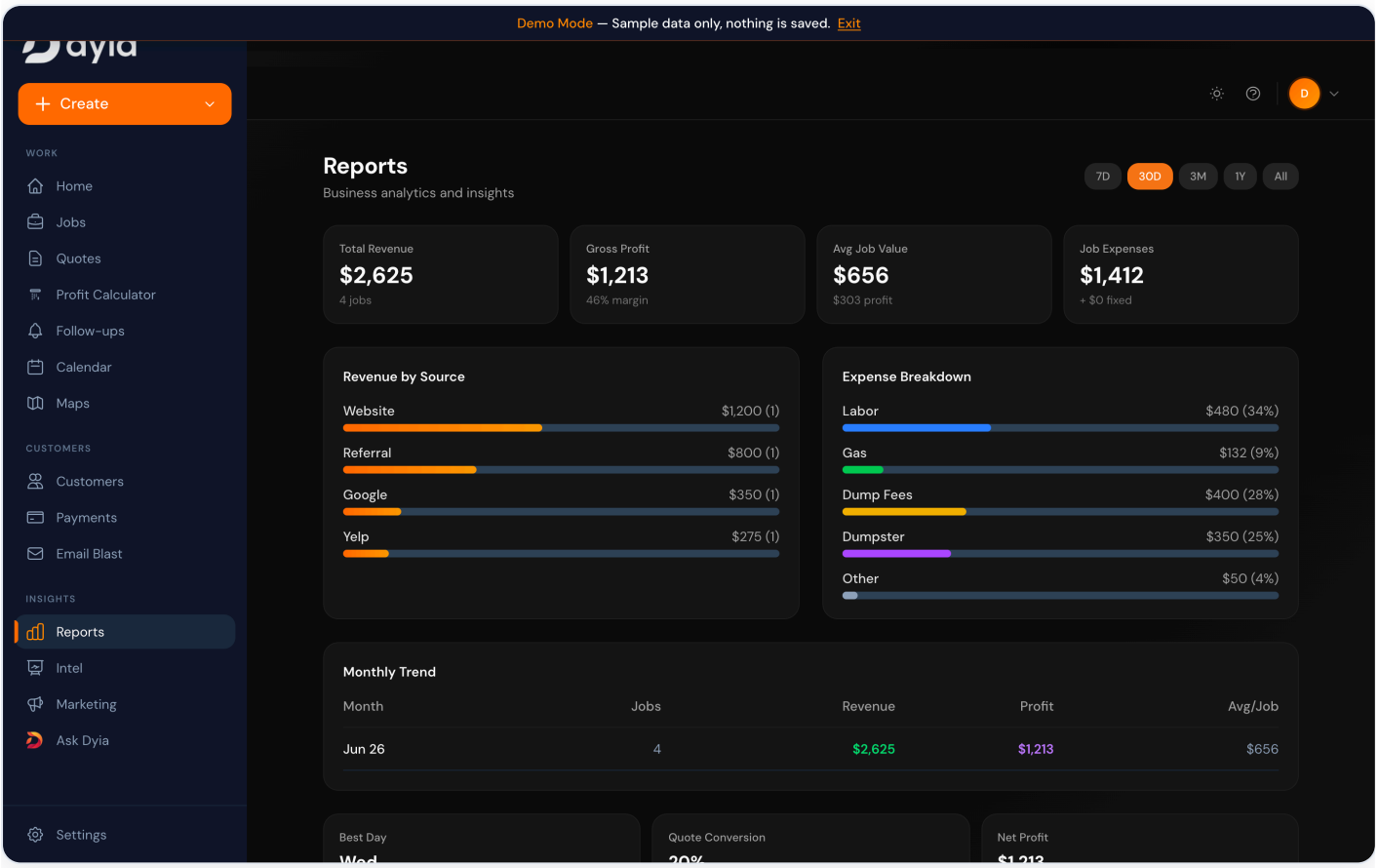
Follow-up pipeline as a drag-and-drop board (Pending / Contacted / Snoozed) with hot/warm/cold prioritization and pipeline value. **Proves:** an explicit status state-machine driving an operational workflow — directly analogous to a contact/lead pipeline.



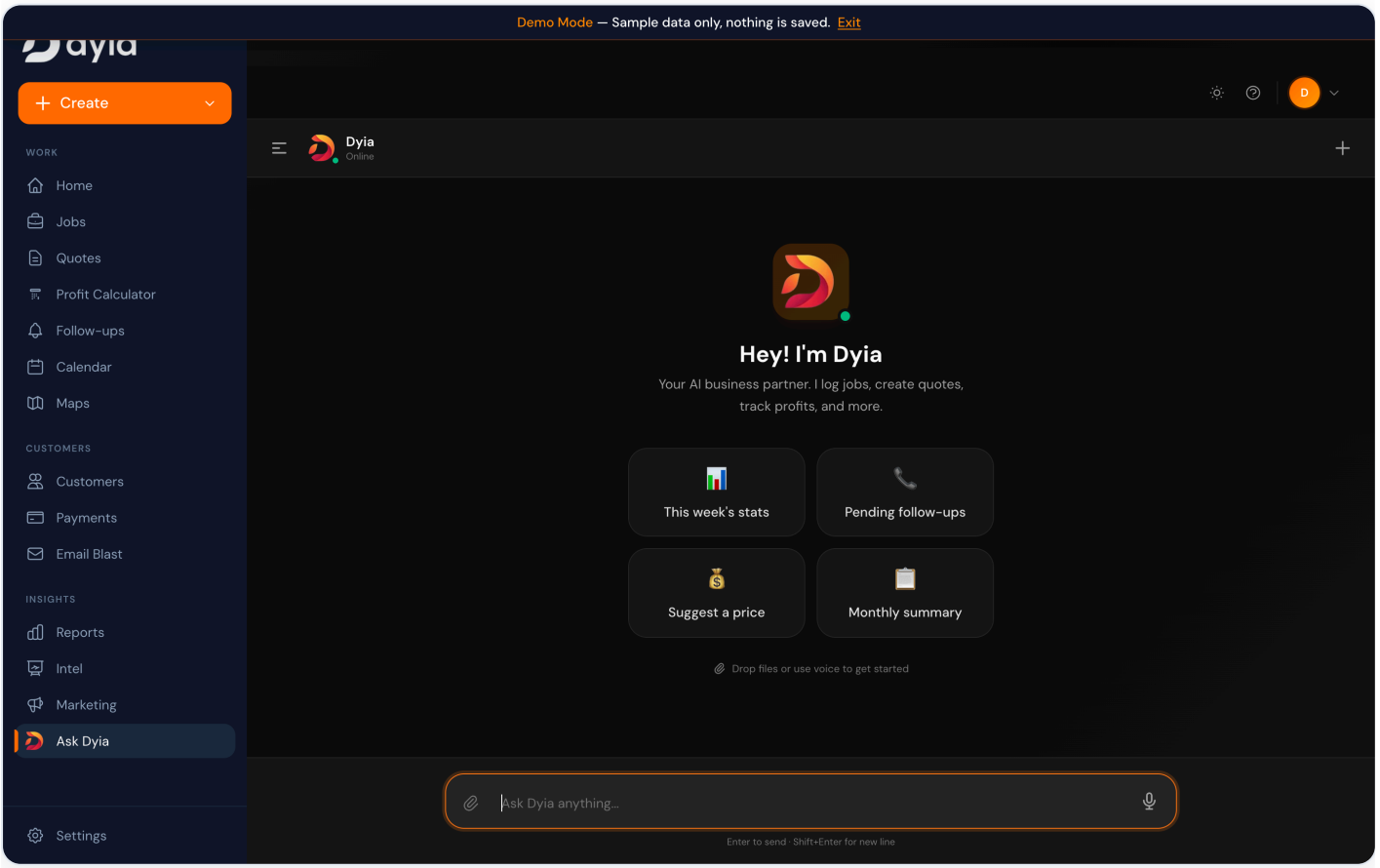
Structured quote builder (customer, line-item pricing, photos, estimate range) with PDF export. **Proves:** form-driven document generation and server-side artifact creation.



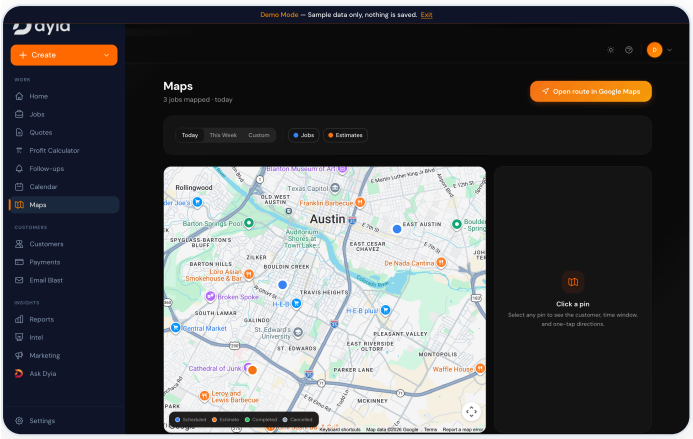
Payments hub: pay links, invoices, and quote/job-linked requests with a transparent fee-and-payout model. **Proves:** a real merchant-payments integration with platform fees, webhook reconciliation, and money-movement state.



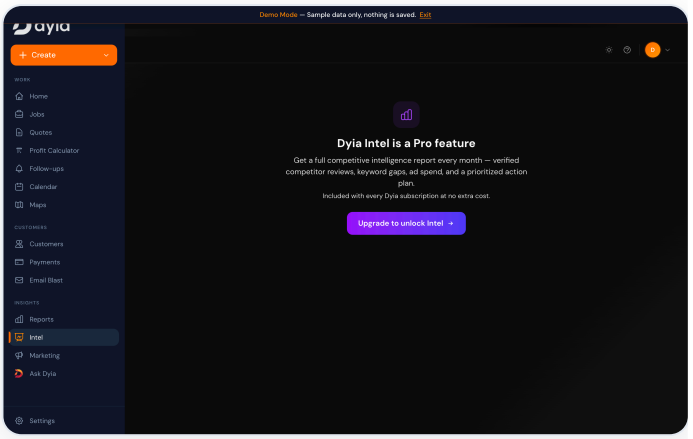
Reporting: revenue by source, expense breakdown, margins, and monthly trend. **Proves:** analytics derived from the operational data model, with time-range selection.



The AI assistant workspace with suggested actions (this week's stats, pending follow-ups, suggest a price, monthly summary). **Proves:** a conversational interface wired into the same backend tools that power the UI.



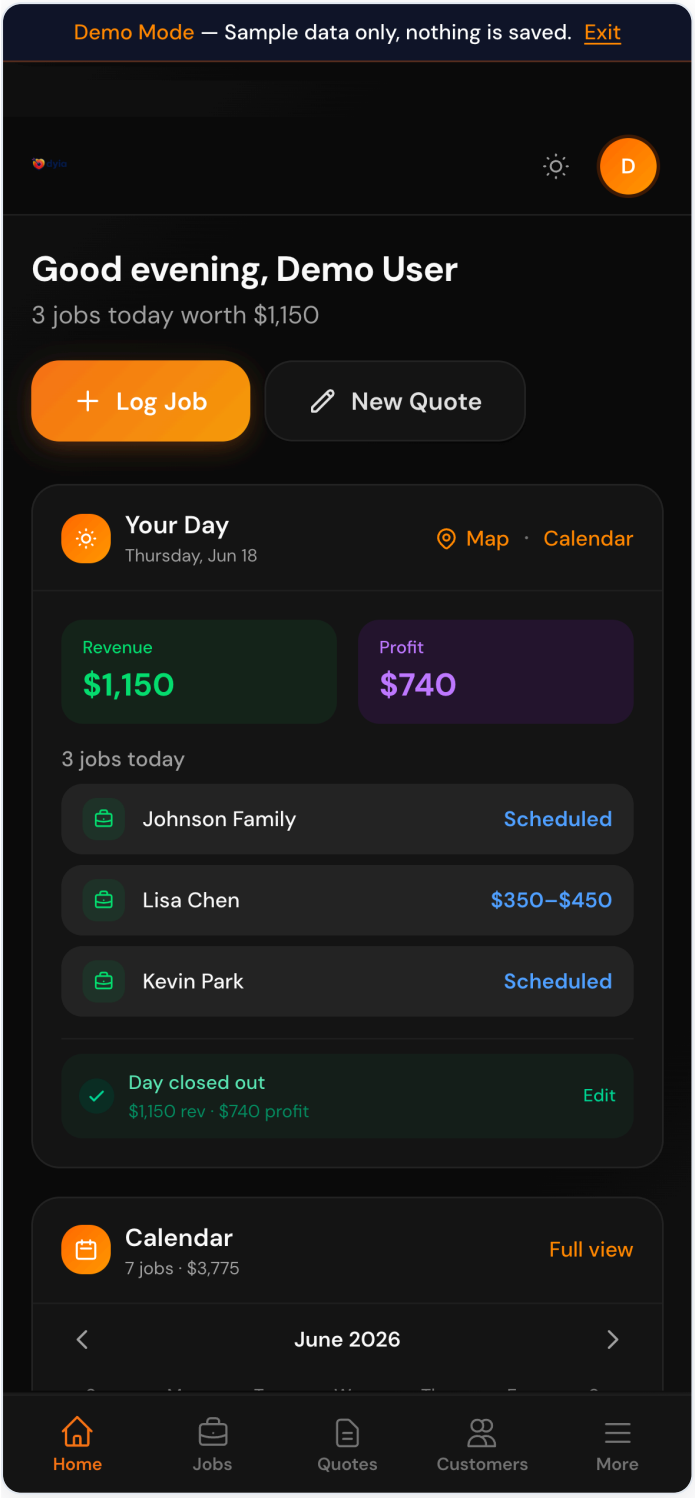
Geocoded jobs on a live map with route planning. **Proves:** third-party geospatial integration tied to records.



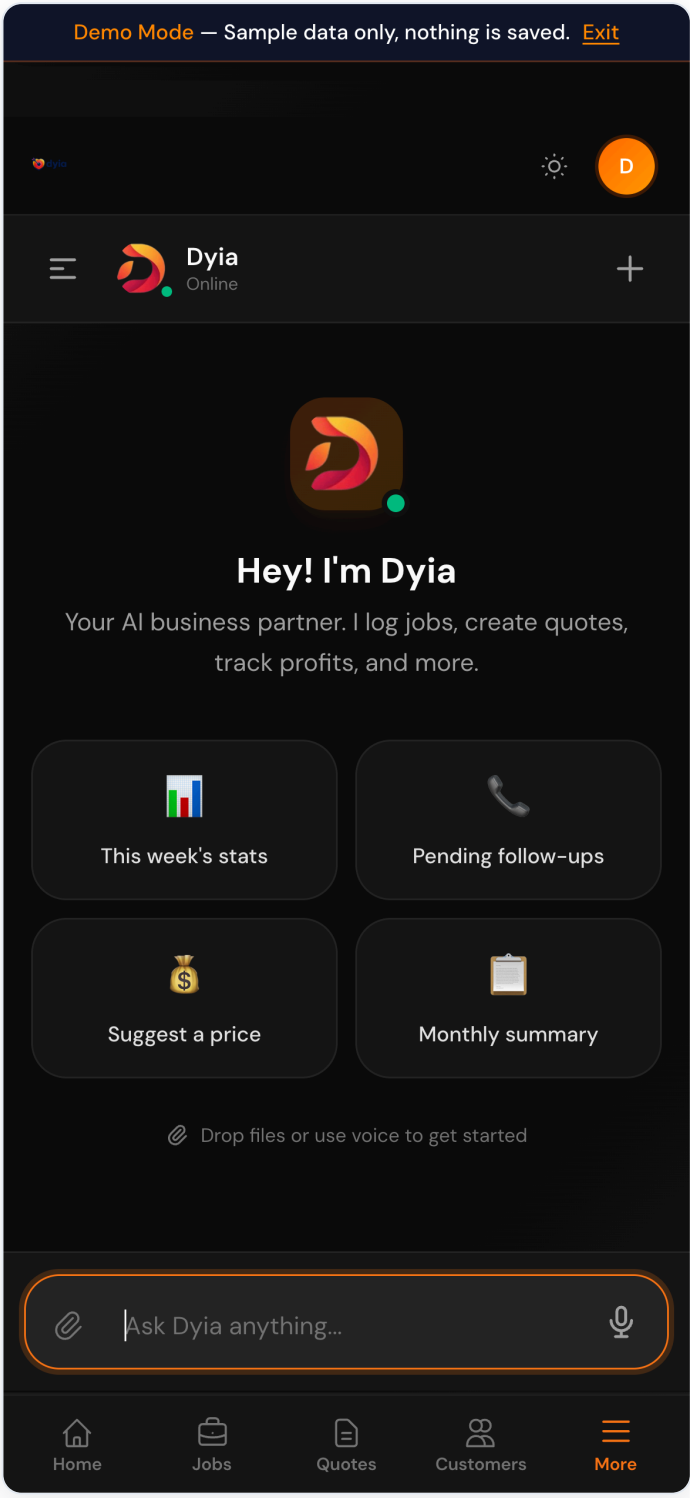
A Pro-gated feature screen. **Proves:** subscription-tier feature gating enforced in the UI.

Responsive / mobile

The same components reflow to a mobile layout with bottom navigation and touch-sized targets — the product is built mobile-first for operators working in the field.



Mobile dashboard — daily view and quick actions.



Mobile AI assistant — full conversational workspace on a phone.

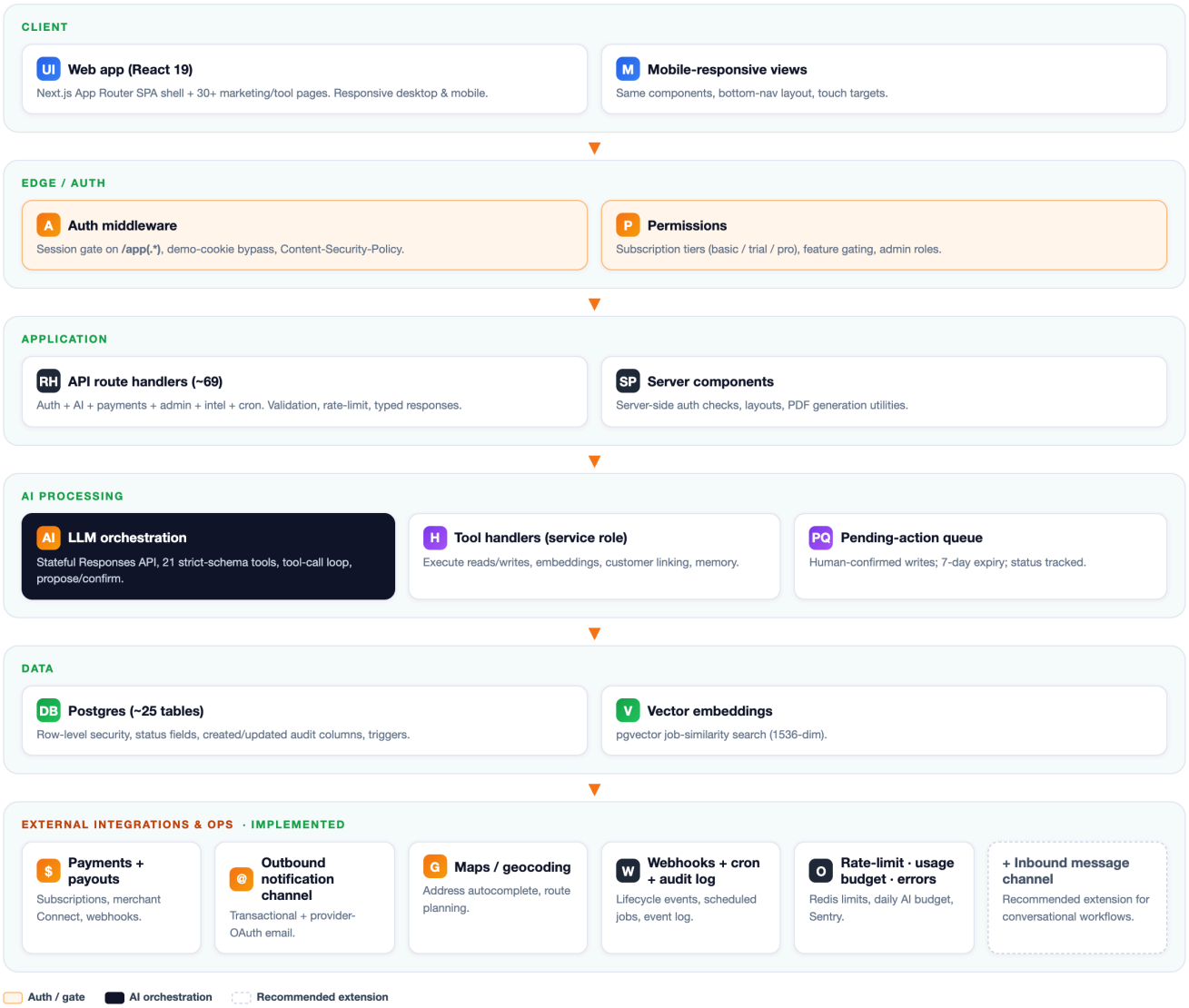
3 Architecture Overview

A single Next.js (App Router) application that serves the public site, the authenticated product, and the API. Server route handlers own privileged operations; the browser talks to the database under row-level security.

Layer	Technology	Notes
Frontend	Next.js (App Router), React 19, TypeScript, Tailwind	Authenticated product is a single-page shell that switches views; 30+ public/marketing/tool pages.
API layer	~69 Next.js route handlers	Auth, AI, payments, admin, intelligence, cron, webhooks. Typed responses, validation, rate limiting.
Auth / permissions	Hosted auth + middleware	Session gate on `/app(.*)`, subscription tiers, admin roles, demo-mode bypass cookie, CSP.
Database / storage	Managed Postgres (Supabase) + pgvector	~25 tables, row-level security, status enums, audit columns, vector embeddings for similarity search.
AI / LLM	OpenAI Responses API	Stateful threads, 21 strict-schema tools, server-side handlers using a service-role DB client.
Integrations	Billing + merchant payouts, email, maps/geocoding, error monitoring, Redis	Subscription checkout, Connect payouts, transactional + OAuth email, address autocomplete, Sentry, Upstash rate limiting.
Async / scheduled	Webhooks + cron + audit log	Signature-verified webhooks, 4 scheduled cron jobs, webhook-event audit table.
Deployment	Vercel (inferred from config)	`vercel.json` declares build, install, and cron schedules.

System Architecture

Actual request and data flow across the platform. Green = implemented today. Dashed = recommended extension.



Real request and data flow. Components shown are present in the codebase; the single dashed node (**inbound message channel**) is labeled as a recommended extension, not an existing feature.

Data-flow note. The browser uses an anon database client carrying the user's auth token, so row-level security applies to client reads. Privileged work — AI writes, webhooks, admin actions — runs in server handlers with a service-role client that intentionally bypasses RLS, keeping trust on the server side.

4 AI System Design

The AI layer is an orchestrator, not a chat wrapper. It turns a natural-language message into typed tool calls, runs a bounded reasoning loop, and routes any state-changing intent through human confirmation.

Where AI calls happen

A dedicated chat route handles the main assistant; additional routes generate dashboard insights, a daily briefing, suggested quick actions, and price suggestions. All calls are server-side; the API key never reaches the browser.

Inputs sent to the model

- The user's message, plus optional image or CSV/text file content as multi-part input.
- System instructions and the 21-tool schema; a prior response id to maintain stateful thread context.
- Resolved tenant identity (the model's tools operate only on the authenticated user's data).

Outputs and how they are stored

- Read tools return structured business data back into the loop (stats, pending follow-ups, forecasts, similar jobs).
- Mutating tools (`propose_job`, `propose_quote`) produce **proposals**, surfaced as preview cards, not silent writes.
- Final assistant text plus tool calls, tool results, token counts, and credit cost are persisted to thread and message tables; usage is logged to a budget ledger.

Guardrails, validation, and review IMPLEMENTED

Control	Mechanism
No silent mutations	Propose/confirm: writes require an explicit user approval step.
Bounded reasoning	Capped tool-call iterations per request prevents runaway loops.
Cost control	Per-request output-token cap; daily spend budget check with alert thresholds.
Abuse control	Per-IP rate limiting (Redis) on the chat endpoint.
Access control	Pro subscription or a positive AI-credit balance required; usage debits a credit ledger.
Schema safety	Strict JSON tool schemas (no additional properties); handler-side field validation.
Determinism	Automatic SDK retries disabled so user-visible outcomes are predictable.

AI Request Lifecycle

How a single user message becomes a guarded, persisted, optionally human-approved action.

1. **User message**
   Free text, optional image or CSV/file context, conversation id, prior response id.
2. **Access & credit gate**
   Pro subscription OR AI-credit balance > 0; per-IP rate limit (30/min); daily spend budget check.
3. **Responses API call**
   Stateful call with system instructions + 21 strict-schema tools; store=true; previous_response_id threads context.
4. **Tool-call loop (≤ 10 iterations)**
   Model selects tools. Read tools return data to the model; the loop continues until a final answer.
5. **Branch: propose vs. direct write**
   Mutating intents (new job / quote) become PROPOSALS, not silent writes. Read + low-risk tools execute directly.
6. **Human confirm (pending action)**
   Proposal is stored and surfaced as a preview card. The user confirms or dismisses before any record is created.
7. **Confirmed write to database**
   On approval, the handler writes the record (service role), links the customer, and may trigger follow-ups / embeddings.
8. **Persist + account**
   Final assistant text saved to thread + messages; tokens, tool calls, credit cost and usage logged.

GUARDRAILS (IMPLEMENTED)

- No automatic SDK retries (deterministic outcomes)
- Max output tokens per response
- Max tool iterations per request
- Strict JSON tool schemas, no extra fields
- Per-IP rate limiting (Redis)
- Daily cost budget + alert thresholds
- Subscription / credit enforcement

PERSISTED PER TURN

- Thread + message rows
- Tool calls & tool results
- Tokens used & credit cost
- Usage / budget ledger
- Cross-thread user memory

The lifecycle of a single message: gate → reason → branch (propose vs. direct) → human confirm → write → persist + account.

> **Honest scope.** Responses are non-streaming by design (deterministic single-shot outcomes). Embeddings power job-similarity search today. A retrieval layer over historical conversations is a natural PROPOSED extension.

5 Workflow / Automation Relevance

Automation-heavy AI systems are built from state, triggers, routing, approvals, logging, and retries. The platform already demonstrates most of these primitives.

Workflow primitive	Status	How it appears in this platform
State management	IMPLEMENTED	Explicit status enums on jobs, quotes, follow-ups, payments, and pending actions.
User roles	IMPLEMENTED	`user / admin / super_admin` plus subscription tiers gating features.
Task / item status	IMPLEMENTED	Follow-up board moves items through pending → contacted → converted / lost / snoozed.
Workflow steps	IMPLEMENTED	Quote → follow-up → convert-to-job → payment is a multi-step lifecycle.
Event triggers	IMPLEMENTED	Signature-verified webhooks; auto-create follow-up when a quote is created.
Routing	IMPLEMENTED	The AI tool-dispatch router selects and executes the correct handler per intent.
Approval flow	IMPLEMENTED	Propose/confirm gate on AI writes; confirm endpoint executes only approved actions.
Pause / resume	PARTIAL	Pending-action queue persists proposed work with a status and 7-day expiry; a user can confirm later or let it lapse.
Human review	IMPLEMENTED	Preview cards for proposed jobs/quotes; admin review of users, payments, beta access.
Logging	IMPLEMENTED	Webhook-event audit table, AI usage/cost ledger, credit transactions, email send logs.
Retries / reconciliation	PARTIAL	A sync-pending route reconciles payments against the provider when a webhook is missed.
Error handling	IMPLEMENTED	React error boundaries plus server-side error monitoring (Sentry).
Notifications	IMPLEMENTED	Transactional email; scheduled reminder and insight emails.
Background jobs	IMPLEMENTED	Four scheduled cron jobs (trial reminders, weekly insights, follow-up reminders, monthly intelligence).

Relevant Extension Pattern. For higher-volume automation, the same primitives extend cleanly to: a durable queue/worker for outbound side-effects with automatic retry/back-off; conversation-level pause/resume flags; and idempotency keys on event handlers. These are additive — the status fields, audit log, and confirm/queue pattern needed to support them already exist. PROPOSED

6 Human-in-the-Loop / Manual Control

The defining design choice of the AI layer is that it does not act unilaterally on anything that mutates the system of record.

What exists today IMPLEMENTED

- **Propose, don't execute.** When the assistant decides to create a job or quote, it emits a proposal rather than writing. The proposal is persisted to a pending-actions table with a status and expiry.
- **Explicit confirmation.** The UI renders a preview card; only on user approval does a dedicated confirm endpoint perform the write (linking the customer, triggering downstream follow-ups).
- **Admin overrides.** An admin role can review and manage users, subscriptions, payments, beta access, and webhook events — manual control over the operational surface.

Proposed manual-control extension PROPOSED

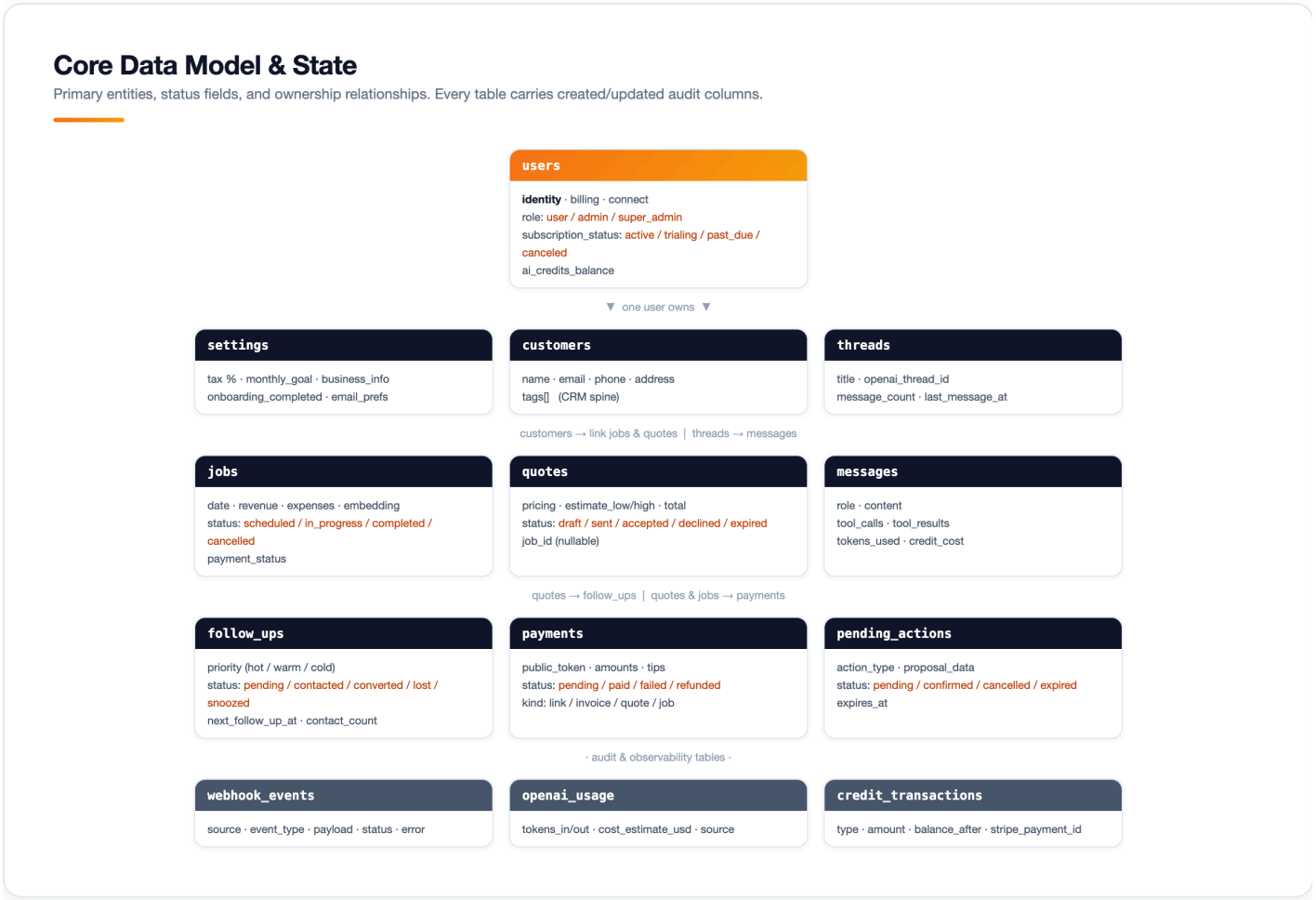
For a conversation/workflow product, the same pattern generalizes into operator takeover. Stated in neutral, channel-agnostic terms:

- The AI acts by default **only while rules allow** (eligible contact state, within rate windows).
- A human can **take over** a conversation at any time.
- While a human is in control, **automation pauses** for that contact — no auto-replies are sent.
- Every state change is **logged** (who took over, when, why).
- Automation **resumes only on an approved condition** (operator hands back, or a timeout/rule is met).
- Contact **statuses and tags** update based on the interaction, feeding routing and follow-up eligibility.

> This extension reuses three things the platform already has: a persisted action/queue with status, an explicit approval gate, and an append-only audit log. The work is wiring a per-contact pause flag and a takeover state into the orchestration layer.

7 Data Model and State

Every entity is tenant-scoped to a user, carries created/updated audit columns, and (where it represents work) a status field that drives the UI and workflow.



Core entities, status enums, and ownership relationships. Audit/observability tables sit alongside the operational model.

Entity	Role	Key status / state
`users`	Identity, billing, payout onboarding, AI credits, role	subscription_status, role
`settings`	Per-tenant business config, onboarding, preferences	onboarding_completed
`customers`	CRM spine; links jobs and quotes	tags[]
`jobs`	Work records with revenue/expense math + embedding	scheduled / in_progress / completed / cancelled; payment_status
`quotes`	Estimates with pricing and optional job link	draft / sent / accepted / declined / expired
`follow_ups`	Lead pipeline with prioritization	pending / contacted / converted / lost / snoozed
`payments`	Pay links / invoices / quote & job payments	pending / paid / failed / refunded / …
`pending_actions`	AI proposals awaiting human confirmation	pending / confirmed / cancelled / expired
`threads / messages`	AI conversation history + tool I/O + cost	is_archived

`webhook_events`	Inbound event audit log	status, error
`openai_usage` / `credit_transactions`	Cost & credit ledgers	type

Relationships: a user owns settings, customers, jobs, quotes, threads, and pending actions; customers link to their jobs and quotes; quotes can spawn follow-ups and convert to jobs; quotes and jobs both link to payments; threads contain messages. Referential cleanup is handled (for example, a quote's job link is nulled if the job is deleted).

8 Production Readiness

An honest assessment. Most production concerns are addressed; remaining gaps are labeled as recommendations rather than claimed.

Area	Status	Evidence / recommendation
Authentication	IMPLEMENTED	Hosted auth + middleware gate on all `/app` routes; webhook-synced user records.
Authorization	IMPLEMENTED	Subscription tiers, admin roles, row-level security on tenant data, server-side service-role isolation.
Validation	IMPLEMENTED	Strict AI tool schemas; payment amount/line-item validation; handler-side checks.
Error handling	IMPLEMENTED	React error boundaries; Sentry on client, server, and edge.
Logging / audit	IMPLEMENTED	Webhook-event log, AI usage ledger, credit transactions, email send logs.
Secrets / config	IMPLEMENTED	Documented `.env.example`; keys read server-side only; live/test-mode guard prevents cross-mode data corruption.
Deployment	IMPLEMENTED	`vercel.json` with build/install commands and four cron schedules.
Schema management	IMPLEMENTED	45+ ordered SQL migrations under `supabase/migrations/`.
API structure	IMPLEMENTED	Consistent route-handler layout by domain; webhook signature verification (svix / provider).
Responsive UI	IMPLEMENTED	Mobile-first layouts verified in capture (see Walkthrough).
Performance	PARTIAL	Rate limiting, cached insights, vector search. Recommend query/index review at scale + caching headers audit.
Security hardening	IMPLEMENTED	CSP, timing-safe demo-token compare, service-role kept server-side, webhook verification.
Automated tests	PARTIAL	Vitest configured with unit tests for subscription, payment-mode, and maps logic. Recommend expanding to API-handler and AI-tool coverage + an e2e smoke suite.

Recommendations (not yet built). Broaden automated test coverage around API handlers and AI tools; add idempotency keys and a durable retry queue for outbound side-effects; formalize a staging environment and migration CI. None block the current product; all are incremental. PROPOSED

9 Relevance to a Conversational Workflow System

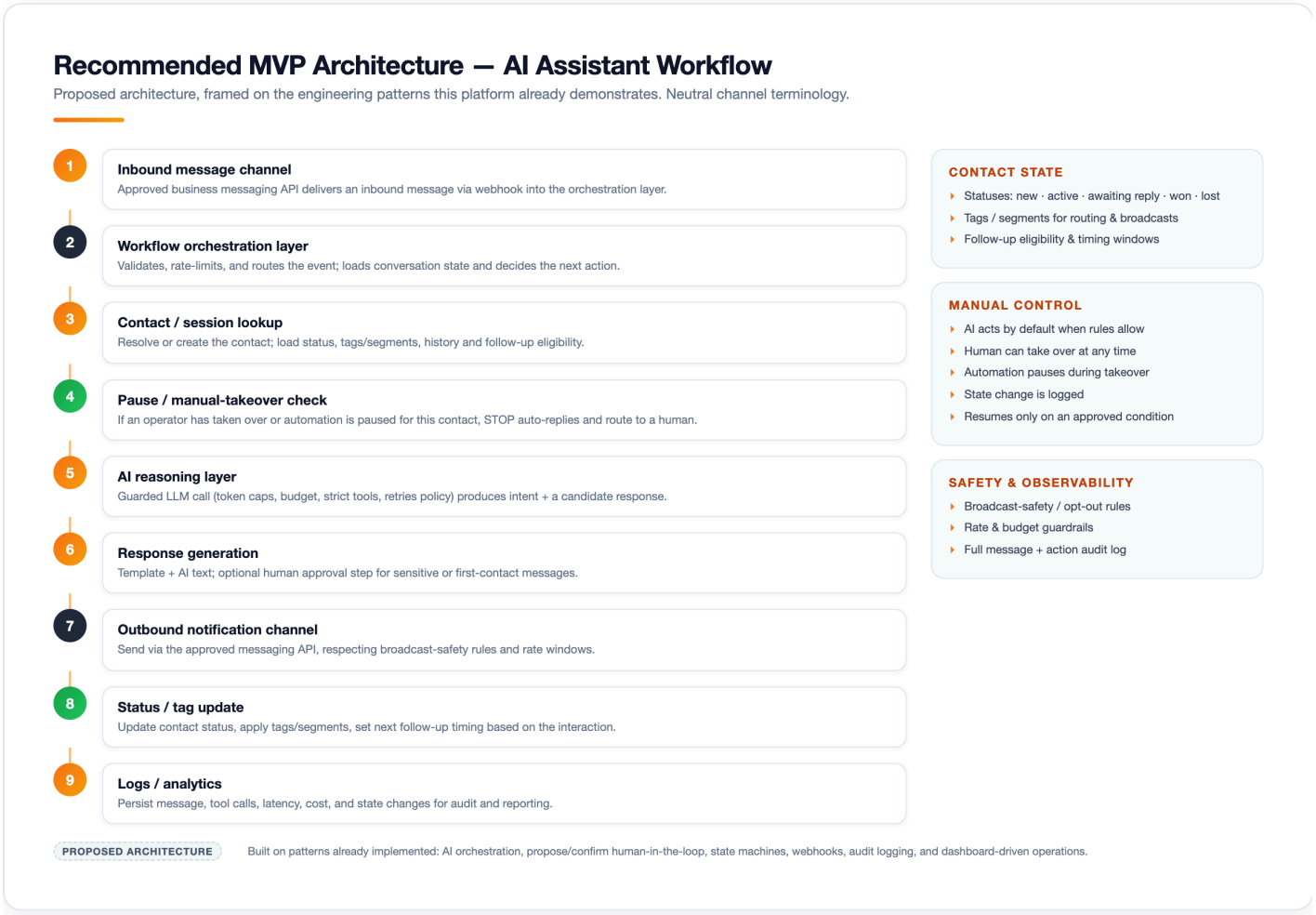
A team building an AI-assisted conversation/workflow system — business messaging integrations, contact/status tracking, manual takeover, follow-up logic, logs and analytics, scaling across operators — needs a specific set of engineering patterns. This platform is not that product, but it demonstrates the same patterns in production.

Requested capability	Pattern this platform demonstrates
AI-assisted conversation workflows	Tool-calling orchestration with a bounded reasoning loop and propose/confirm control.
Business messaging integrations	Signature-verified webhooks in, transactional/OAuth notification channels out, provider reconciliation.
Workflow orchestration	Multi-step lifecycles (quote → follow-up → job → payment) with explicit status transitions.
Contact / status tracking	CRM entity with tags, plus status-driven follow-up pipeline (hot/warm/cold).
Manual takeover	Human-in-the-loop approval gate and admin overrides; generalizes to per-contact pause/takeover.
Follow-up logic	Auto-created follow-ups, snooze/next-contact timing, scheduled reminder jobs.
Logs & analytics	Audit tables, usage/cost ledgers, and a reporting layer over operational data.
Scaling across operators/accounts	Multi-tenant model with per-user scoping, RLS, roles, and subscription tiers.

> In short: the platform shows **AI orchestration, dashboard-driven operations, backend state management, real API integrations, user-facing workflow design, and production SaaS architecture** — the exact foundations a conversational workflow system is built on.

10 Recommended MVP Architecture

A proposed, lean architecture for an AI assistant workflow system, expressed in neutral channel terminology and grounded in the patterns above. This is a recommendation, not an existing feature of the platform.



Inbound message channel → orchestration → contact/session lookup → pause/manual-takeover check → AI reasoning → response generation → outbound notification channel → status/tag update → logs/analytics.

State & eligibility

- Contact statuses (new, active, awaiting reply, won, lost).
- Tags / segments for routing and broadcasts.
- Follow-up eligibility and timing windows.

Safety & control

- Manual takeover with automatic pause/resume.
- Broadcast-safety / opt-out rules before any send.
- Rate and budget guardrails on the AI layer.
- Full message + state-change audit log.

Why this is credible. Each block maps to something already shipped here: orchestration and tool routing (AI chat), pause/approval (propose/confirm + pending-actions queue), status/tags (follow-ups + customers), and logging (audit + usage ledgers). The MVP is an assembly of proven parts, not a research project. PROPOSED

11 One-Page Summary

What it is

A production, multi-tenant AI SaaS for service businesses: track jobs and profit, build and send quotes, run a follow-up pipeline, collect payments, and operate the whole thing through a tool-calling AI assistant that asks for confirmation before it changes anything.

What Dev built / owns

The full stack — product UI, ~69 API route handlers, the AI orchestration layer (21 tools + propose/confirm), a ~25-table relational data model with 45+ migrations, integrations for auth, billing, merchant payouts, email, maps, and monitoring, plus the operational tooling (admin, webhooks, cron jobs, rate/budget guardrails).

What the architecture demonstrates

AI orchestration with human-in-the-loop control, backend state machines with audit logging, real third-party API integrations, multi-tenant SaaS security (auth, RLS, roles, tiers), and production deployment hygiene.

Why it is relevant to AI SaaS & automation workflows

The same primitives a conversational workflow system needs — orchestration, contact/status tracking, manual takeover, follow-up logic, logs/analytics, and per-account scaling — are already shipped here in a different product. The recommended MVP is an assembly of these proven parts.

Technical highlights

- Tool-calling AI with strict schemas, bounded loops, and no silent writes.
- Rate limiting, daily cost budget, and credit accounting on the AI layer.
- Status-driven workflows (jobs, quotes, follow-ups, payments, pending actions).
- Webhook verification, service-role isolation, RLS, and an append-only audit trail.